A better alternative, supported by many database systems today, is to use the normalized schema, and additionally store the join or *account* and *depositor* as a materialized view. (Recall that a materialized view is a view whose result is stored in the database, and brought up to date when the relations used in the view are updated.) Like denormalization, using materialized view does have space and time overheads; however, it has the advantage that keeping the view up to date is the job of the database system, not the application programmer.

7.10.4 Other Design Issues

There are some aspects of database design that are not addressed by normalization, and can thus lead to bad database design. We give examples here; obviously, such designs should be avoided.

Consider a company database, where we want to store earnings of companies in different years. A relation *earnings*(*company-id*, *year*, *amount*) could be used to store the earnings information. The only functional dependency on this relation is *company-id*, *year* $\rightarrow$ *amount*, and the relation is in BCNF.

An alternative design is to use multiple relations, each storing the earnings for a different year. Let us say the years of interest are 2000, 2001, and 2002; we would then have relations of the form *earnings-2000*, *earnings-2001*, *earnings-2002*, all of which are on the schema (*company-id*, *earnings*). The only functional dependency here on each relation would be *company-id* $\rightarrow$ *earnings*, so these relations are also in BCNF.

However, this alternative design is clearly a bad idea—we would have to create a new relation every year, and would also have to write new queries every year, to take each new relation into account. Queries would also be more complicated since they may have to refer to many relations.

Yet another way of representing the same data is to have a single relation *company-year*(*company-id*, *earnings-2000*, *earnings-2001*, *earnings-2002*). Here the only functional dependencies are from *company-id* to the other attributes, and again the relation is in BCNF. This design is also a bad idea since it has problems similar to the previous design—namely we would have to modify the relation schema and write new queries, every year. Queries would also be more complicated, since they may have to refer to many attributes.

Representations such as those in the *company-year* relation, with one column for each value of an attribute, are called **crosstabs**; they are widely used in spreadsheets and reports and in data analysis tools. While such representations are useful for display to users, for the reasons just given, they are not desirable in a database design. SQL extensions have been proposed to convert data from a normal relational representation to a crosstab, for display.

7.11 Summary

- We showed pitfalls in database design, and how to systematically design a database schema that avoids the pitfalls. The pitfalls included repeated information and inability to represent some information.

- We introduced the concept of functional dependencies, and showed how to reason with functional dependencies. We laid special emphasis on what dependencies are logically implied by a set of dependencies. We also defined the notion of a canonical cover, which is a minimal set of functional dependencies equivalent to a given set of functional dependencies.
- We introduced the concept of decomposition, and showed that decompositions must be lossless-join decompositions, and should preferably be dependency preserving.
- If the decomposition is dependency preserving, given a database update, all functional dependencies can be verifiable from individual relations, without computing a join of relations in the decomposition.
- We then presented Boyce–Codd Normal Form (BCNF); relations in BCNF are free from the pitfalls outlined earlier. We outlined an algorithm for decomposing relations into BCNF. There are relations for which there is no dependency-preserving BCNF decomposition.
- We used the canonical covers to decompose a relation into 3NF, which is a small relaxation of the BCNF condition. Relations in 3NF may have some redundancy, but there is always a dependency-preserving decomposition into 3NF.
- We presented the notion of multivalued dependencies, which specify constraints that cannot be specified with functional dependencies alone. We defined fourth normal form (4NF) with multivalued dependencies. Section C.1.1 of the appendix gives details on reasoning about multivalued dependencies.
- Other normal forms, such as PJNF and DKNF, eliminate more subtle forms of redundancy. However, these are hard to work with and are rarely used. Appendix C gives details on these normal forms.
- In reviewing the issues in this chapter, note that the reason we could define rigorous approaches to relational-database design is that the relational data model rests on a firm mathematical foundation. That is one of the primary advantages of the relational model compared with the other data models that we have studied.

Review Terms

- Atomic domains
- First normal form
- Pitfalls in relational-database design
- Functional dependencies
- Superkey
- F holds on R
- R satisfies F
- Trivial functional dependencies
- Closure of a set of functional dependencies
- Armstrong's axioms

- Closure of attribute sets
- Decomposition
- Lossless-join decomposition
- Legal relations
- Dependency preservation
- Restriction of F to R_i
- Boyce–Codd normal form (BCNF)
- BCNF decomposition algorithm
- Canonical cover
- Extraneous attributes
- Third normal form
- 3NF decomposition algorithm
- Multivalued dependencies
- Fourth normal form
- restriction of a multivalued dependency
- Project-join normal form (PJNF)
- Domain-key normal form
- E-R model and normalization
- Universal relation
- Unique-role assumption
- Denormalization

Exercises

7.1 Explain what is meant by *repetition of information* and *inability to represent information*. Explain why each of these properties may indicate a bad relational-database design.

7.2 Suppose that we decompose the schema $R = (A, B, C, D, E)$ into

$$(A, B, C)$$
$$(A, D, E)$$

Show that this decomposition is a lossless-join decomposition if the following set F of functional dependencies holds:

$$A \rightarrow BC$$
$$CD \rightarrow E$$
$$B \rightarrow D$$
$$E \rightarrow A$$

7.3 Why are certain functional dependencies called *trivial* functional dependencies?

7.4 List all functional dependencies satisfied by the relation of Figure 7.21.

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2
a_2	b_1	c_1
a_2	b_1	c_3

Figure 7.21 Relation of Exercise 7.4.

7.5 Use the definition of functional dependency to argue that each of Armstrong's axioms (reflexivity, augmentation, and transitivity) is sound.

7.6 Explain how functional dependencies can be used to indicate the following:

- A one-to-one relationship set exists between entity sets *account* and *customer*.
- A many-to-one relationship set exists between entity sets *account* and *customer*.

7.7 Consider the following proposed rule for functional dependencies: If $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, then $\alpha \rightarrow \gamma$. Prove that this rule is *not* sound by showing a relation *r* that satisfies $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, but does not satisfy $\alpha \rightarrow \gamma$.

7.8 Use Armstrong's axioms to prove the soundness of the union rule. (*Hint*: Use the augmentation rule to show that, if $\alpha \rightarrow \beta$, then $\alpha \rightarrow \alpha\beta$. Apply the augmentation rule again, using $\alpha \rightarrow \gamma$, and then apply the transitivity rule.)

7.9 Use Armstrong's axioms to prove the soundness of the decomposition rule.

7.10 Use Armstrong's axioms to prove the soundness of the pseudotransitivity rule.

7.11 Compute the closure of the following set *F* of functional dependencies for relation schema $R = (A, B, C, D, E)$.

$$A \rightarrow BC$$
$$CD \rightarrow E$$
$$B \rightarrow D$$
$$E \rightarrow A$$

List the candidate keys for *R*.

7.12 Using the functional dependencies of Exercise 7.11, compute B^+.

7.13 Using the functional dependencies of Exercise 7.11, compute the canonical cover F_c.

7.14 Consider the algorithm in Figure 7.22 to compute α^+. Show that this algorithm is more efficient than the one presented in Figure 7.7 (Section 7.3.3) and that it computes α^+ correctly.

7.15 Given the database schema $R(a, b, c)$, and a relation r on the schema R, write an SQL query to test whether the functional dependency $b \rightarrow c$ holds on relation r. Also write an SQL assertion that enforces the functional dependency. Assume that no null values are present.

7.16 Show that the following decomposition of the schema *R* of Exercise 7.2 is not a lossless-join decomposition:

$$(A, B, C)$$
$$(C, D, E)$$

Hint: Give an example of a relation *r* on schema *R* such that

$$\Pi_{A,B,C}(r) \bowtie \Pi_{C,D,E}(r) \neq r$$

```
result := ∅;
/* fdcount is an array whose ith element contains the number
   of attributes on the left side of the ith FD that are
   not yet known to be in α+ */
for i := 1 to |F| do
   begin
      let β → γ denote the ith FD;
      fdcount [i] := |β|;
   end
/* appears is an array with one entry for each attribute. The
   entry for attribute A is a list of integers. Each integer
   i on the list indicates that A appears on the left side
   of the ith FD */
for each attribute A do
   begin
      appears [A] := NIL;
      for i := 1 to |F| do
         begin
            let β → γ denote the ith FD;
            if A ∈ β then add i to appears [A];
         end
   end
addin (α);
return (result);

procedure addin (α);
for each attribute A in α do
   begin
      if A ∉ result then
         begin
            result := result ∪ {A};
            for each element i of appears[A] do
               begin
                  fdcount [i] := fdcount [i] − 1;
                  if fdcount [i] := 0 then
                     begin
                        let β → γ denote the ith FD;
                        addin (γ);
                     end
               end
         end
   end
```

Figure 7.22 An algorithm to compute α^+.

7.17 Let $R_1, R_2, \ldots, R_n$ be a decomposition of schema U. Let $u(U)$ be a relation, and let $r_i = \Pi_{R_I}(u)$. Show that

$$u \subseteq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

7.18 Show that the decomposition in Exercise 7.2 is not a dependency-preserving decomposition.

7.19 Show that it is possible to ensure that a dependency-preserving decomposition into 3NF is a lossless-join decomposition by guaranteeing that at least one schema contains a candidate key for the schema being decomposed. (*Hint*: Show that the join of all the projections onto the schemas of the decomposition cannot have more tuples than the original relation.)

7.20 List the three design goals for relational databases, and explain why each is desirable.

7.21 Give a lossless-join decomposition into BCNF of schema R of Exercise 7.2.

7.22 Give an example of a relation schema R' and set F' of functional dependencies such that there are at least three distinct lossless-join decompositions of R' into BCNF.

7.23 In designing a relational database, why might we choose a non-BCNF design?

7.24 Give a lossless-join, dependency-preserving decomposition into 3NF of schema R of Exercise 7.2.

7.25 Let a *prime* attribute be one that appears in at least one candidate key. Let α and β be sets of attributes such that $\alpha \rightarrow \beta$ holds, but $\beta \rightarrow \alpha$ does not hold. Let A be an attribute that is not in α, is not in β, and for which $\beta \rightarrow A$ holds. We say that A is *transitively dependent* on α. We can restate our definition of 3NF as follows: A relation schema R is in 3NF with respect to a set F of functional dependencies if there are no nonprime attributes A in R for which A is transitively dependent on a key for R.

Show that this new definition is equivalent to the original one.

7.26 A functional dependency $\alpha \rightarrow \beta$ is called a **partial dependency** if there is a proper subset γ of α such that $\gamma \rightarrow \beta$. We say that β is *partially dependent* on α. A relation schema R is in **second normal form** (2NF) if each attribute A in R meets one of the following criteria:

- It appears in a candidate key.
- It is not partially dependent on a candidate key.

Show that every 3NF schema is in 2NF. (*Hint*: Show that every partial dependency is a transitive dependency.)

7.27 Given the three goals of relational-database design, is there any reason to design a database schema that is in 2NF, but is in no higher-order normal form? (See Exercise 7.26 for the definition of 2NF.)

7.28 Give an example of a relation schema R and a set of dependencies such that R is in BCNF, but is not in 4NF.

7.29 Explain why 4NF is a normal form more desirable than BCNF.

7.30 Explain how dangling tuples may arise. Explain problems that they may cause.

Bibliographical Notes

The first discussion of relational-database design theory appeared in an early paper by Codd [1970]. In that paper, Codd also introduced functional dependencies, and first, second, and third normal forms.

Armstrong's axioms were introduced in Armstrong [1974]. Ullman [1988] is an easily accessible source of proofs of soundness and completeness of Armstrong's axioms. Ullman [1988] also provides an algorithm for testing for lossless-join decomposition for general (nonbinary) decompositions, and many other algorithms, theorems, and proofs concerning dependency theory. Maier [1983] discusses the theory of functional dependencies.Graham et al. [1986] discusses formal aspects of the concept of a legal relation.

BCNF was introduced in Codd [1972]. The desirability of BCNF is discussed in Bernstein et al. [1980a]. A polynomial-time algorithm for BCNF decomposition appears in Tsou and Fischer [1982], and can also be found in Ullman [1988]. Biskup et al. [1979] gives the algorithm we used to find a lossless-join dependency-preserving decomposition into 3NF. Fundamental results on the lossless-join property appear in Aho et al. [1979a].

Multivalued dependencies are discussed in Zaniolo [1976]. Beeri et al. [1977] gives a set of axioms for multivalued dependencies, and proves that the authors axioms are sound and complete. Our axiomatization is based on theirs. The notions of 4NF, PJNF, and DKNF are from Fagin [1977], Fagin [1979], and Fagin [1981], respectively.

Maier [1983] presents the design theory of relational databases in detail. Ullman [1988] and Abiteboul et al. [1995] present a more theoretic coverage of many of the dependencies and normal forms presented here. See the bibliographical notes of Appendix C for further references to literature on normalization.

PART 3

Object-based Databases and XML

Several application areas for database systems are limited by the restrictions of the relational data model. As a result, researchers have developed several data models to deal with these application domains. In this part, we study the object-oriented data model and the object-relational data model. In addition, we study XML, a language that can represent data that is less structured than that of the other data models.

The object-oriented data model, described in Chapter 8, is based on the object-oriented-programming language paradigm, which is now in wide use. Inheritance, object-identity, and encapsulation (information hiding), with methods to provide an interface to objects, are among the key concepts of object-oriented programming that have found applications in data modeling. The object-oriented data model also supports a rich type system, including structured and collection types. While inheritance and, to some extent, complex types are also present in the E-R model, encapsulation and object-identity distinguish the object-oriented data model from the E-R model.

The object-relational model, described in Chapter 9, combines features of the relational and object-oriented models. This model provides the rich type system of object-oriented databases, combined with relations as the basis for storage of data. It applies inheritance to relations, not just to types. The object-relational data model provides a smooth migration path from relational databases, which is attractive to relational database vendors. As a result, the SQL:1999 standard includes a number of object-oriented features in its type system, while continuing to use the relational model as the underlying model.

The XML language was initially designed as a way of adding markup information to text documents, but has become important because of its applications in data exchange. XML provides a way to represent data that have nested structure, and furthermore allows a great deal of flexibility in structuring of data, which is important for certain kinds of nontraditional data. Chapter 10 describes the XML language, and then presents different ways of expressing queries on data represented in XML, and transforming XML data from one form to another.